

## Trabajo Práctico N° 5

### Tipos de Datos y Sistemas de Tipos

**Lectura:** Scott Cap. 7, Sebesta Cap. 6

#### Conceptos

1. ¿A qué nos referimos cuando hablamos de tipo de dato? ¿Qué es un sistema de tipos? ¿Cuáles son los principales criterios de evaluación de un lenguaje de programación que se ven afectados por el sistema de tipos? ¿Por qué?
2. Defina qué significa que un lenguaje sea fuertemente tipado. ¿Qué desventajas tiene un lenguaje que no es fuertemente tipado? Ejemplifique.
3. ¿Cuál es la diferencia entre un error del lenguaje y un error de aplicación? ¿Qué relación tienen estos conceptos con la noción de lenguaje fuertemente tipado?
4. Defina la noción del grado de dureza de una característica del sistema de tipos. Seleccione dos características diferentes del sistema de tipos y, para cada una de ellas, indique restricciones que influyen en su grado de dureza.
5. Analice las ventajas y desventajas de:
  - a. Tipado estático vs. Tipado dinámico.
  - b. Conversión implícita vs. Conversión explícita.
  - c. Alto grado de dureza vs Bajo grado de dureza.
6. Describa las principales decisiones de diseño que un lenguaje puede tomar al especificar los atributos de las cadenas de caracteres, arreglos y arreglos asociativos. ¿Cómo pueden impactar estas decisiones en el chequeo de tipos del lenguaje?
7. Explique en qué consisten los tipos ordinal, enumerado y subrango. ¿De qué manera las decisiones de diseño asociadas a estos tipos pueden afectar a los chequeos de tipos en un lenguaje?
8. ¿Qué chequeos de tipos puede realizar un lenguaje en torno al tipo *unión*? ¿Qué sucede en caso de *unión discriminada*?
9. Explique a qué se denomina *coerción*. ¿Cuáles son las desventajas de tener coerciones respecto al chequeo de tipos? ¿Cómo influyen en el grado de dureza del sistema de tipos?
10. Indique claramente qué efectos en la determinación de tipos puede tener un lenguaje de programación que cuenta simultáneamente con coerción, sobrecarga y expresiones mixtas. En caso de generar problemas, ¿qué puede hacer el lenguaje para evitarlos?
11. Indique en qué aspectos el alcance dinámico afecta al sistema de tipos.
12. Muestre gráficamente las distintas formas de polimorfismo y explique cada una de ellas. ¿A qué criterios de evaluación afecta que el lenguaje ofrezca alguna forma de polimorfismo?
13. ¿Qué ventaja ofrece incorporar polimorfismo paramétrico en un lenguaje con tipado estático (como Java) frente a uno que no lo incorpore? Justifique su respuesta.
14. Los siguientes ítems muestran conjuntos de características relacionadas a los lenguajes de programación. Para cada uno, indique si es posible y/o conveniente incorporar las características mencionadas, de manera conjunta, en un lenguaje de programación real. Justifique adecuadamente.
  - a. Tipado dinámico, expresiones mixtas, coerciones, sobrecarga de métodos.
  - b. Tipado estático, alcance estático, polimorfismo por inclusión, polimorfismo paramétrico.
  - c. Tipado estático, alcance dinámico, polimorfismo por inclusión, sobrecarga de operadores.
15. Los siguientes ítems describen cuatro lenguajes de programación hipotéticos. ¿Cuál considera que estaría más cerca de ser fuertemente tipado? ¿Por qué?
  - a. Un lenguaje con tipado estático, duro en su sistema de tipos en general, con tipo unión.
  - b. Un lenguaje con tipado dinámico, herencia simple, sin expresiones mixtas ni coerciones.

- c. Un lenguaje con tipado estático, expresiones mixtas, sobrecarga dependiente del contexto.
- d. Un lenguaje con tipado estático, expresiones mixtas, polimorfismo paramétrico y herencia simple.

## Casos de Estudio e Investigación

16. Evalúe las expresiones “hola” \* 4 y “hola”.longitud <3 en JavaScript. ¿Qué resultado obtiene en cada caso? ¿Por qué?
17. En base a lo reportado en el ejercicio anterior, ¿es JavaScript un lenguaje fuertemente tipado? Justifique su respuesta. Luego, indique dos cambios que realizaría en JavaScript para que su sistema de tipos sea más duro.
18. Explique por qué Java no es considerado un lenguaje fuertemente tipado. Ilustre con un ejemplo.
19. ¿Qué lenguaje tiene un sistema de tipos más duro respecto a expresiones mixtas, Pascal o Python? ¿Por qué? ¿Y respecto a las declaraciones de tipo?
20. Realice un análisis comparativo de la dureza del sistema de tipos entre JavaScript y Python respecto a expresiones mixtas.
21. Investigue las reglas de coerción que aplica JavaScript sobre expresiones mixtas con los operadores +, -, \*, /, % y ==. Muestre ejemplos. ¿Cómo impactan estas decisiones en el chequeo de tipos? ¿Qué ventajas y desventajas puede apreciar?
22. Considere las siguientes sentencias en Java:
  - a. `double v1 = 4.0 / (5 / 10);`
  - b. `double v2 = 4.0 / (5.0 / 10);`¿Qué diferencias ve entre a) y b)? ¿Qué valores quedarán almacenados en v1 y v2 respectivamente luego de ejecutar esas sentencias? ¿Por qué son diferentes?
23. Considere el programa en Python que se muestra a la derecha, y la siguiente expresión:

`f1() + f1(1) + f1('a') + f1(1, 0) + f1('a', 3)`

  - a. ¿Por qué la expresión genera un error? ¿Qué característica (o características) del sistema de tipos restringe el lenguaje para que genere un error?
  - b. Imagine un lenguaje similar a Python que incorpore la (o las) características identificadas en el inciso anterior. ¿Cuál sería el resultado (o los resultados posibles) de la expresión anterior? ¿Qué puede decir acerca de la dureza en cuanto a expresiones mixtas de este lenguaje?
24. Compare las facilidades provistas por Python y Java para soportar arreglos. Presente ejemplos mostrando las diferencias.
25. Go ofrece cuatro tipos estructurados principales: arrays, slices, structs y maps. Para cada uno, identifique al menos una decisión de diseño relevante que toma el lenguaje y busque un lenguaje de programación donde esa misma decisión haya sido tomada de forma diferente, explicando el contraste.
26. ¿Qué formas de polimorfismo soporta Java, Pascal y Python?
27. ¿Por qué no tendría sentido el uso de polimorfismo paramétrico en un lenguaje como Python? ¿Lo mismo vale para cualquier lenguaje con tipado dinámico?
28. Compare el grado de dureza del sistema de tipos de Python, Java y Pascal respecto a la sobrecarga de métodos.
29. ¿Qué lenguaje tiene un sistema de tipos más duro respecto al polimorfismo por inclusión, Java o Python? ¿Por qué? ¿Y respecto al polimorfismo por sobrecarga?
30. Analice, en lo que se refiere al sistema de tipos, la función módulos desarrollada en Python y el método módulos en Java que se muestran debajo. Indique explícitamente qué características del sistema de tipos son relevantes y en qué grado de dureza están consideradas en cada lenguaje.

| Python                                                                                                                   | Java                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>def módulos(lista):     output = []     for item in lista:         output.append(item % 2)      return output</pre> | <pre>public List&lt;Integer&gt; módulos(List&lt;Integer&gt; lista){     List&lt;Integer&gt; output = new ArrayList&lt;&gt;();     for (Integer item : lista) {         output.add(item % 2);     }      return output; }</pre> |

31. Java y Python utilizan dos enfoques muy distintos para realizar el chequeo de tipos. Investigue cómo funciona el chequeo de tipos en cada lenguaje. En Java, diferencie el comportamiento sobre tipos primitivos y sobre objetos, considerando el rol de la herencia y el polimorfismo. En Python, investigue cómo se determina si una operación sobre un objeto es válida. ¿En qué momento se detectan los errores de tipo en cada lenguaje?
32. Analice el siguiente código Python que se muestra debajo.

```
def calcular_descuento(precio: float, porcentaje: int) -> float:
    return precio * (1 - porcentaje / 100)

def aplicar_descuentos(precios: list[float], descuento: int) -> list[float]:
    return [calcular_descuento(p, descuento) for p in precios]
```

Investigue en qué consisten los *type hints* (o *type annotations*) en Python y cómo impactan en el chequeo de tipos del lenguaje. ¿Cambia el comportamiento del intérprete? ¿Qué ventajas brindan para el desarrollo? Para complementar su respuesta, puede explorar la herramienta *mypy* (enlace).